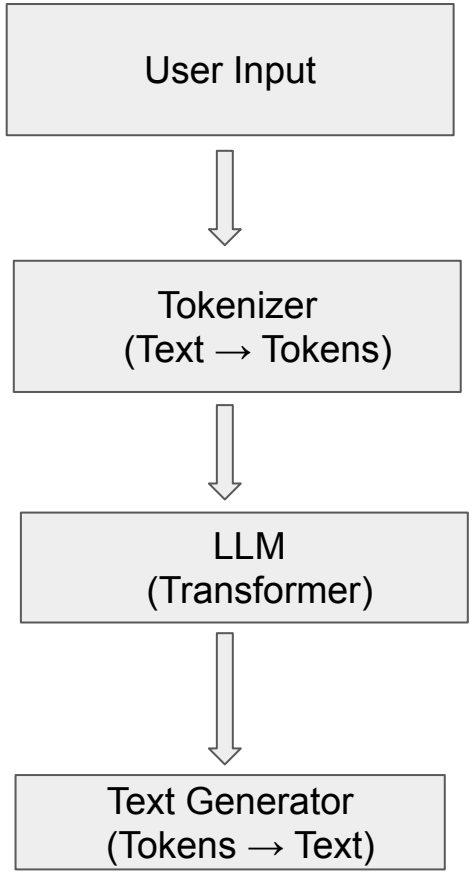
RAG

What is LLM?

- A Large Language Model (LLM) is an AI program that can **understand and create text**.
- It is trained on a **huge amount of data**, so it learns patterns in language.
- LLMs use the **Transformer model**, which helps the system focus on important words using **self-attention**.
- They can do many tasks like:
  - Answering questions
  - Summarizing text
  - Translating languages
- Example: **OpenAI's GPT-3**
  - Has **175 billion parameters** (learning points)
  - Can take **large inputs** (up to 100k tokens in some versions)

What is LLM?

- ➔ In simpler terms , an LLM is a computer program that learns to **understand and use human language** by studying many examples.
- ➔ The **better the training data**, the better the LLM learns.
- ➔ So, developers choose **good-quality and carefully selected data** to train it well.



Why do LLMs need tokenization instead of reading full sentences directly?

Limitations of LLMs

Newton School of Technology

- **Not always updated:**
  They only know information up to their last training date, so they may give wrong answers about recent events.

- **Hallucinations:**
  Sometimes they give answers that sound correct but are actually wrong or confusing.

- **Not great for very specific topics:**
  For detailed questions (like company rules), they may give only general answers.

- **No clear sources:**
  They don’t show where the information comes from, so it is hard to check accuracy.

- **Slow to update:**
  Adding new information requires a lot of time and computing power.

- **May guess answers:**
  Even when they don’t know, they may still give an answer instead of saying “I don’t know.”

What is RAG(Retrieval-Augmented Generation)

Newton School of Technology

What is RAG?

- **RAG = Retrieval-Augmented Generation**
- It is a method that helps LLMs give **better and more accurate answers**.

How RAG Works

- First, it **retrieves** helpful information from documents or databases.
- Then, the LLM **generates** the final answer using both this information and its own knowledge.

Why RAG is Useful

- Gives **more accurate** answers
- Reduces **wrong or made-up** responses (hallucinations)
- Helps with **specific topics** because it uses real external data

Why is RAG Important?

Newton School of Technology

Why LLMs Can Be Problematic

- LLMs sometimes give answers with **too much confidence**, even when they are wrong or missing information.
- This causes:
  - Outdated or incorrect answers
  - Loss of trust
  - Unreliable chatbot behavior

How RAG Helps

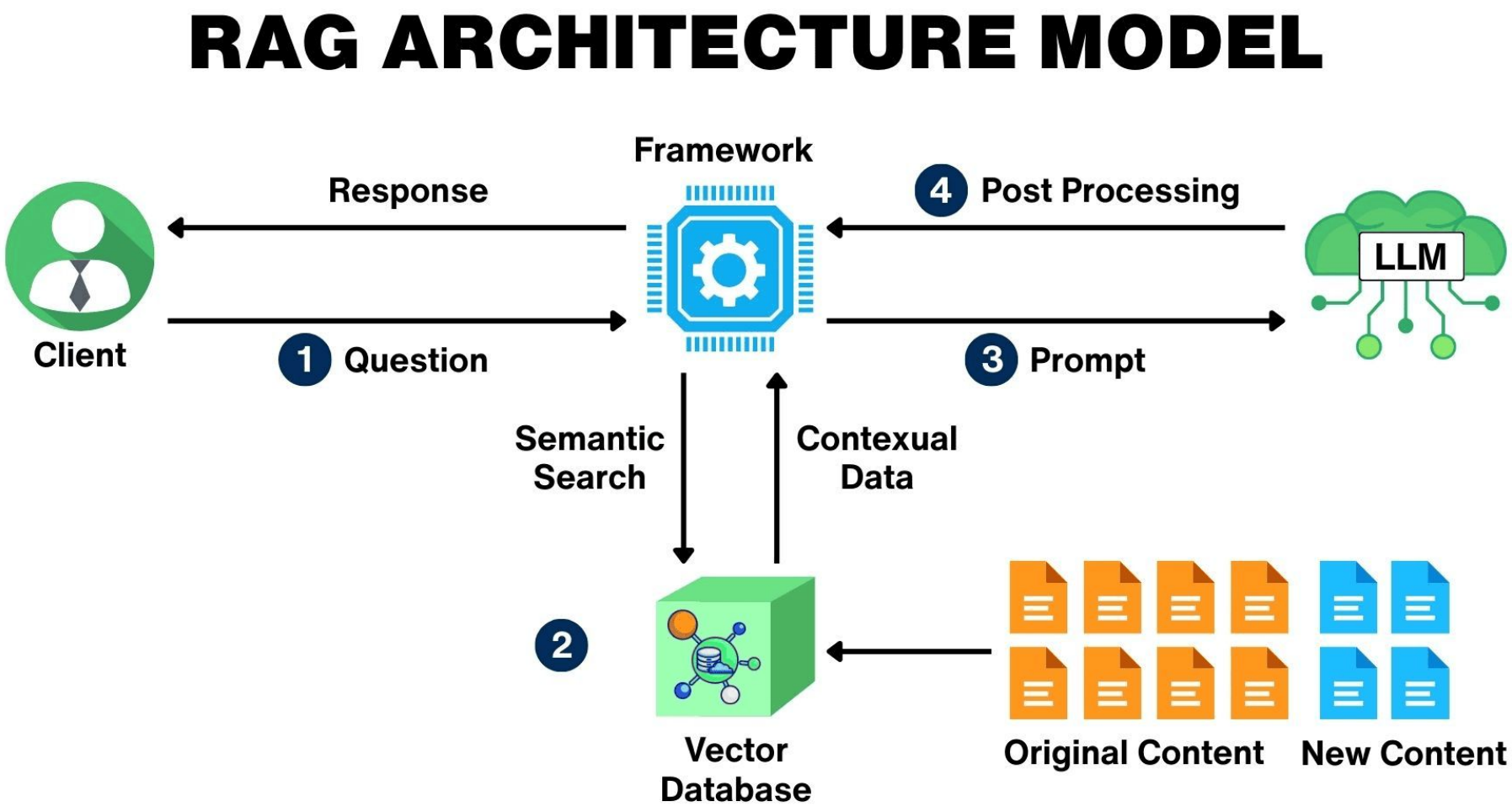
- RAG makes the LLM **fetch correct information** from trusted sources before answering.
- This ensures the response is **accurate, reliable, and up-to-date**.

Benefits for Organizations

- More **control** over what the AI says
- Ensures the AI uses **approved and authoritative knowledge**

What does RAG fix that LLMs alone cannot?

RAG Architecture



How does RAG Works?

1. User Query:
User asks a question.

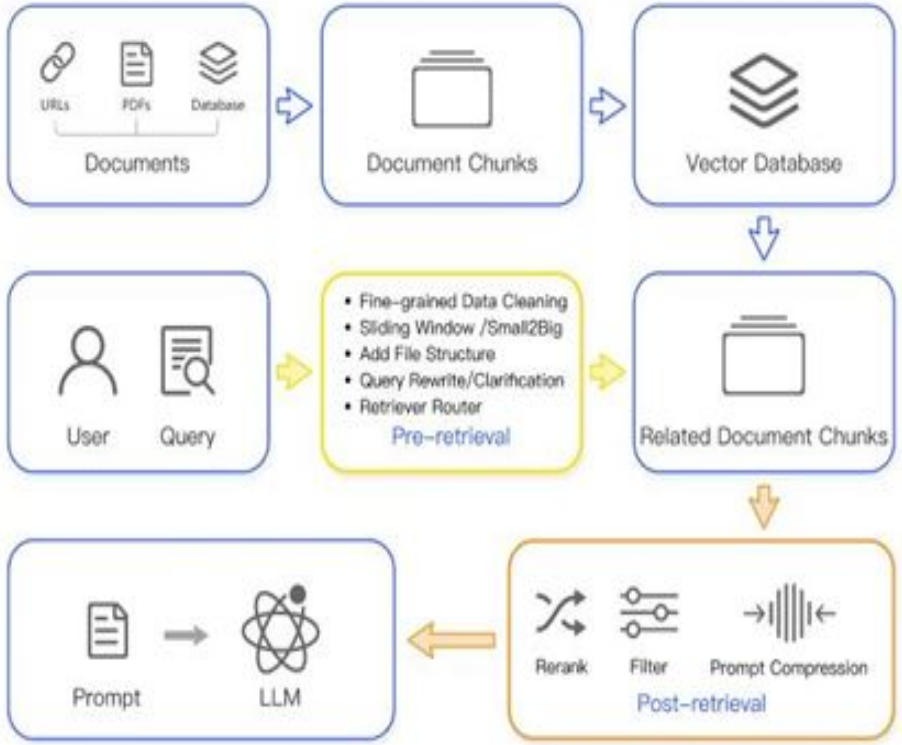
2. Embedding Creation:
The query is converted into vector form.

3. Retrieve Information:
The system searches a vector database to find the most relevant document chunks.

4. Rank & Select:
The best-matching information is chosen.

5. Generate Answer:
The LLM uses the retrieved context + the query to produce an accurate response.

Query → Embeddings → Retrieve → Rank → Generate



RAG Components

- RAG is an AI method that allows a model to **use information from outside its training data**.
- It combines two strengths:
  - **Retrieval:** Finding useful facts
  - **Generation:** Creating the final answer

How It Works

- A **retriever** searches through documents or databases to find relevant information.
- This information is then added to the LLM's knowledge, helping it give **more accurate and relevant answers**.

Which part reduces hallucinations — retrieval or generation?

RAG Components

1. External data
2. Retriever
3. Ranker
4. Generator(LLM)

RAG Components

Newton School of Technology

External Data

External data refers to any new data that does not come from the LLM's original training dataset. It can be sourced from APIs, databases, document repositories, or other storage systems. This data may be stored in various formats such as files, database records, or long-form text.

Vector Embeddings

- ML models **cannot understand raw text**, so everything must be converted into **numbers**.
- **Embeddings** are created by neural networks to turn words or data into **vectors** (lists of numbers).
- These vectors represent the **meaning** of the text in a multi-dimensional space.
- Because of embeddings, ML models can understand patterns in language and work with non-numerical data.

RAG Components

Newton School of Technology

Example:

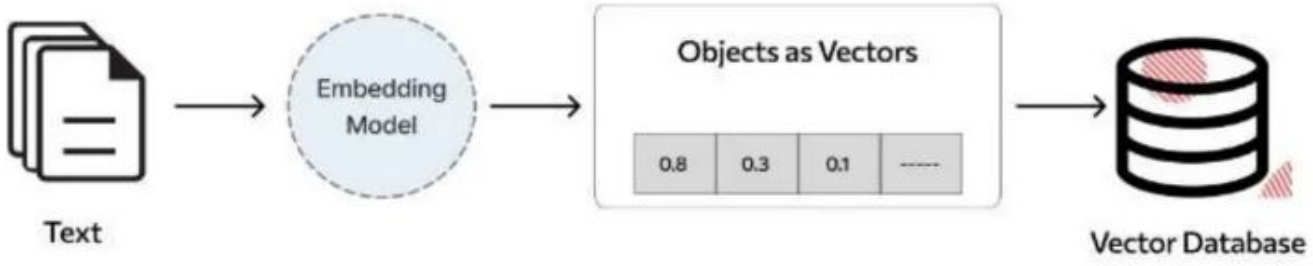
```
The Conference (Horror, 2023, Movie)          →  The Conference (1.2,
2023, 20.0)

Tales from the Crypt (Horror, 1989, TV Show,
Season 7)                                     →  Tales from the Crypt
(1.2, 1989, 36.7)
```

- In this example, the first number in each vector corresponds to a specific genre.
- Because both items share the same genre (Horror), an ML model can identify their similarity.
- By using embeddings, the model can discover deeper relationships between items in a dataset.

Why store embeddings instead of raw text?

RAG Components

- A **Vector Database** stores embeddings — which are numerical representations of text (words, phrases, documents).
- It helps find **similar information quickly** by comparing these vectors.
- In RAG systems, it helps the model **retrieve the most relevant information** before generating an answer.
- Large documents are **broken into small chunks** so they can be searched more easily.
- Examples include **Chroma, Pinecone, Weaviate, and Elasticsearch**.



RAG Components

Newton School of Technology

RAG Retriever

- The retriever takes the user's question and turns it into a **vector** (a set of numbers).
- It then compares this vector with the vectors stored in the **vector database** to find the **most relevant information**.

What It Does

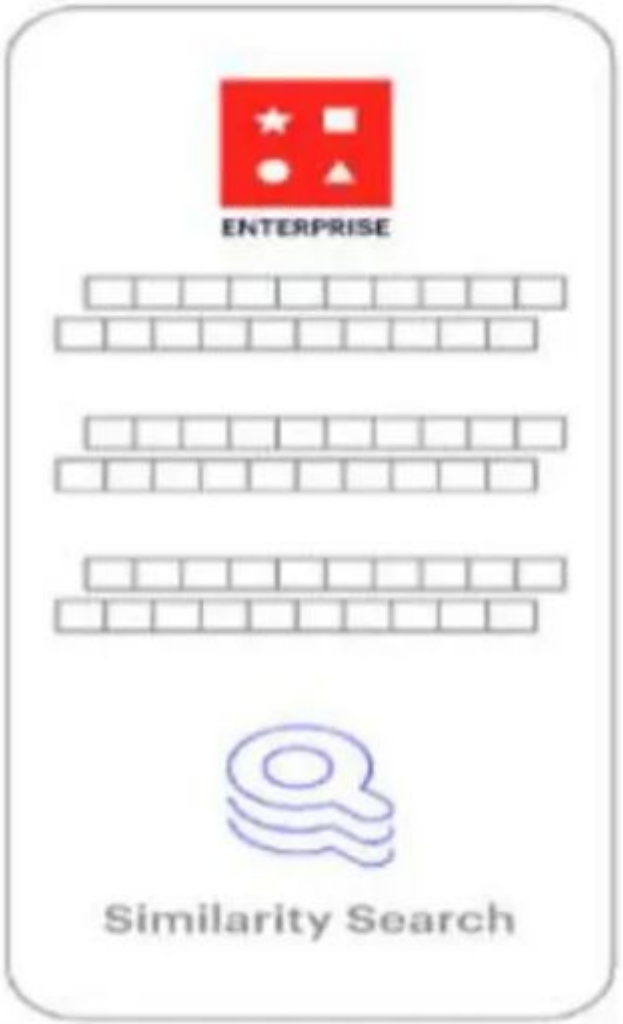
- Quickly finds and pulls out the **right documents or data** from large collections.

Example

- In an HR chatbot:
  If an employee asks, *"How much annual leave do I have?"*
  The retriever finds the **leave policy** and the employee's **leave records**.

Why It Works Well

- It **ranks** the most relevant documents using vector math, ensuring the answer is accurate.



Why is a ranker needed even after the retriever provides results?

RAG Components

Newton School of Technology

RAG Ranker:

- The ranker checks the retrieved information and **decides which pieces are most important**.
- It **assigns scores** so the most relevant data appears first.

Example

- In an HR chatbot:
  If someone asks, *"How much annual leave do I have?"*
  The ranker puts the **leave policy** and the **employee's leave records** at the top because they are the most useful.

Adding Context to the LLM

- The ranker adds the selected important information to the **LLM's prompt**.
- This helps the LLM give **clear, correct, and context-aware answers**.

RAG Components

Newton School of Technology

RAG Generator

- The RAG generator is basically an **LLM (like GPT)**.

What It Does

- Uses the retrieved and ranked information.
- Combines that information with the user's question.
- Creates the **final answer**.

Why It's Important

- Makes sure the answer matches the user's question.
- Includes **correct information** from external sources so the response is reliable.

RAG Components

Newton School of Technology

Updating External Data

To keep the system's information **fresh and accurate**, documents and their embeddings must be updated regularly.

How Updates Happen

1. Real-Time Updates

- New information is added **immediately** as it becomes available.
- Keeps the system **fully up-to-date** at all times.

2. Batch Updates

- Information is updated **periodically** (daily, weekly, etc.).
- Useful for **large datasets** or when instant updates are not needed.

A Simple RAG Chat Application

Newton School of Technology

Install Required Libraries

```
pip install langchain chromadb sentence-transformers openai
```

Load Embedding Model & Sample Documents

```
from sentence_transformers import SentenceTransformer

# Embedding model
embed_model = SentenceTransformer("all-MiniLM-L6-v2")

# Knowledge base documents (can be replaced with PDFs or text files)
documents = [
    "RAG reduces hallucinations by grounding LLM responses in external
data.",
    "Vector databases store embeddings for fast semantic search.",
    "Retrievers fetch top relevant chunks based on vector similarity.",
]
embeddings = embed_model.encode(documents)
```

Initialize Vector Database

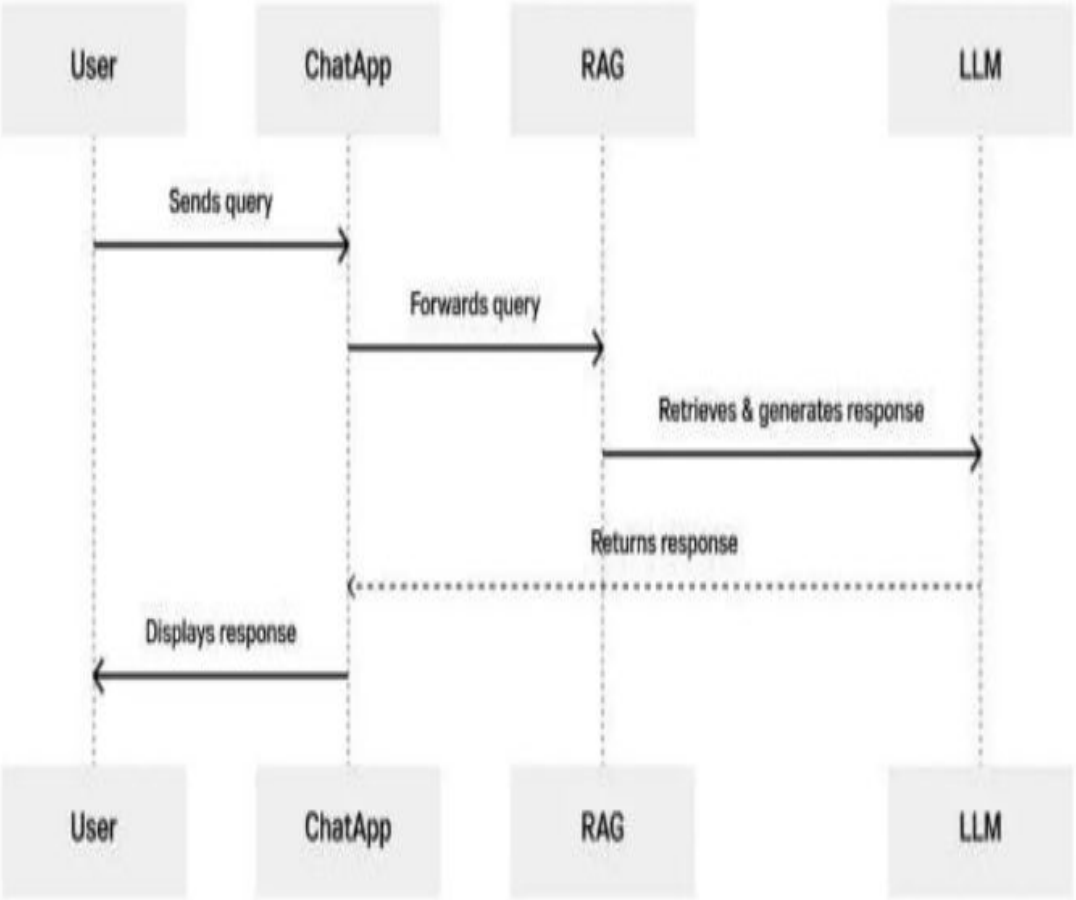
```
from langchain.vectorstores import Chroma

db = Chroma.from_embeddings(documents, embeddings)
```

RAG Chat Application

Newton School of Technology

RAG Chat Function (Query → Retrieve → Generate Answer)

```python
from openai import OpenAI

client = OpenAI()

def rag_chat(query):
    # Step 1: Embed user query
    q_emb = embed_model.encode([query])[0]

    # Step 2: Retrieve top matching chunks
    results = db.similarity_search_by_vector(q_emb, k=2)
    context = "\n".join([doc.page_content for doc in results])

    # Step 3: Build prompt with retrieved context
    prompt = f"""
Use the following context to answer the question.

Context:
{context}

Question: {query}
Answer:
"""

    # Step 4: Generate answer using LLM
    response = client.chat.completions.create(
        model="gpt-4o-mini",
        messages=[{"role": "user", "content": prompt}]
    )

    return response.choices[0].message["content"]
```



RAG Chat Application

Newton School of Technology

Run the RAG Chatbot

```python
while True:
    user_input = input("User: ")
    if user_input.lower() in ["exit", "quit"]:
        break

    answer = rag_chat(user_input)
    print("\nRAG Bot:", answer, "\n")
```

The ChatBot Output Looks like :

```
==================================================
RAG Chatbot is ready! Type 'exit' or 'quit' to stop.
==================================================

User: Hii! what does RAG reduces?

RAG Bot: RAG reduces hallucinations in language model responses by grounding them in external data.

User: what do vector databases do?

RAG Bot: Vector databases store embeddings, which are numerical representations of data, to enable fast s
emantic search and retrieval. They allow for efficient similarity searches based on these embeddings, hel
ping to find relevant chunks of data quickly.

User: quit
Goodbye!
```

Understanding RAG Architecture

1. **User Sends Query**
   - The process starts when the user sends a query or message to the chat application.
2. **Chat App Forwards Query**
   - The chat application (Chat App) sends the user's query to the Retrieval-Augmented Generation (RAG) model for processing.
3. **RAG Retrieves & Generates Response**
   - The RAG model:
     - Combines retrieval and generation capabilities.
     - Retrieves relevant information from a large corpus.
     - Processes the query and retrieved data using the LLM to produce a coherent, contextually relevant response.
4. **LLM Returns Response**
   - The generated response is sent back from the RAG model to the chat application.

5. **Chat App Displays Response**
   - The chat application displays the generated response to the user, completing the interaction.

Challenge:

Newton School of Technology

Task:
After retrieving results, filter out chunks that have similarity > 0.90 among themselves.

```
import numpy as np

unique_results = []
for r in results:
    is_duplicate = False
    for u in unique_results:
        sim = np.dot(r.vector, u.vector) / (np.linalg.norm(r.vector) *
np.linalg.norm(u.vector))
        if sim > 0.90:
            is_duplicate = True
            break
    if not is_duplicate:
        unique_results.append(r)

context = "\n".join([doc.page_content for doc in unique_results])
```

Why can’t we rely only on fine-tuning to fix LLM hallucinations?

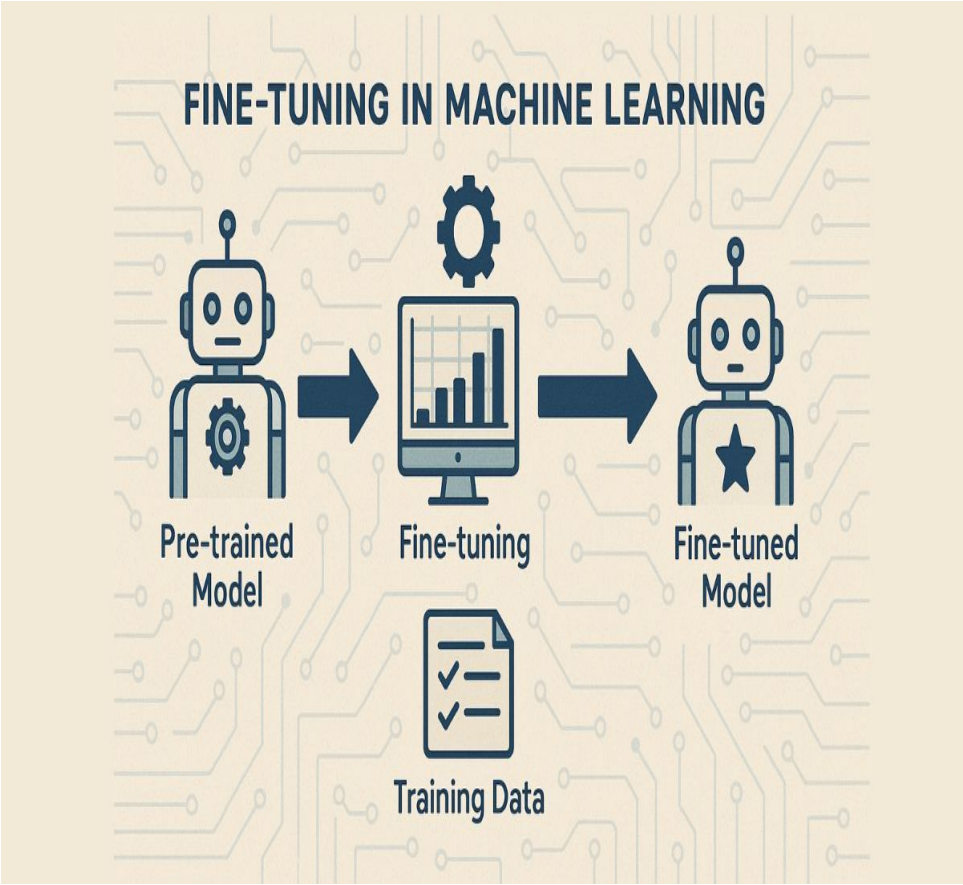
RAG vs Fine-Tuning

RAG (Retrieval-Augmented Generation):

- Retrieves relevant info from external sources to answer questions.
- Flexible: can handle new data by updating the database.
- Cheaper: no need to retrain the model.
- Great for dynamic or factual knowledge.
- Like a student who looks up books to answer questions.

Fine-Tuning:

- Trains the model directly on your specific dataset.
- Accurate for the data it was trained on.
- Needs retraining for new data.
- More expensive and resource-intensive.
- Like a student who memorizes all answers from a textbook.

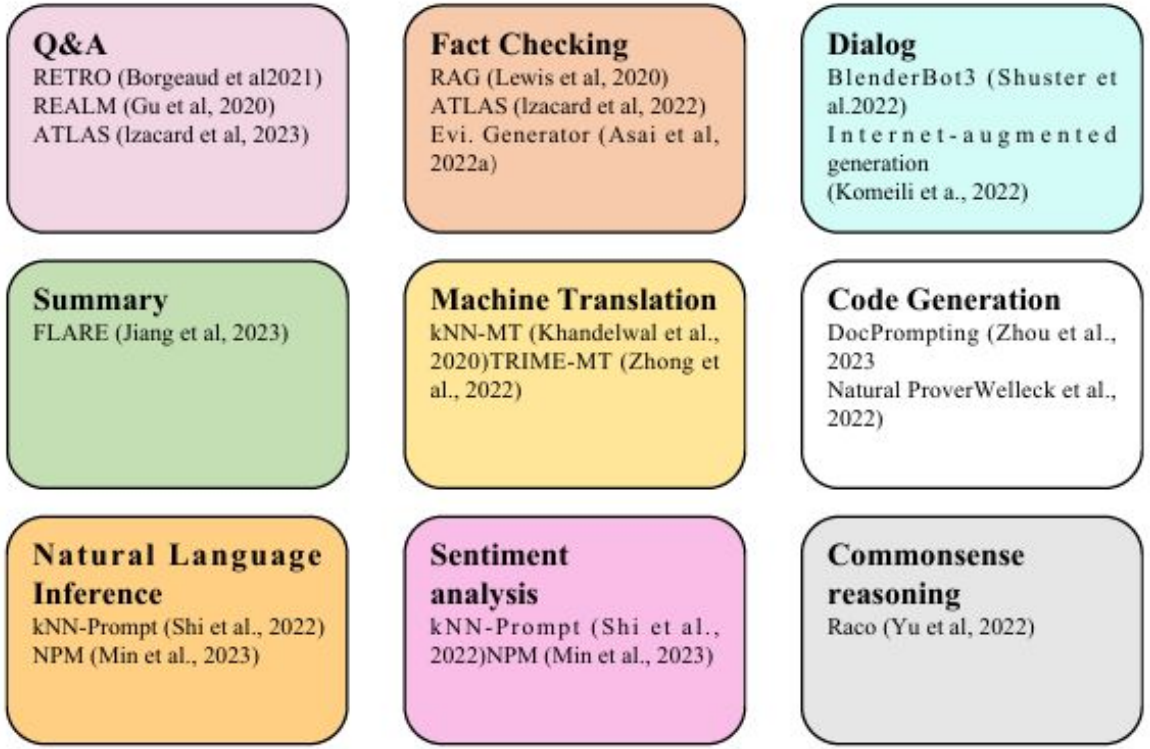


RAG Benefits

- **More Relevant Responses:** Uses external knowledge for better context.
- **Higher Quality Output:** Improves accuracy and reliability.
- **Versatile:** Works across tasks without fine-tuning.
- **Efficient Retrieval:** Minimizes need for labeled data.
- **Real-Time Updates:** Keeps information current.
- **More Trustworthy:** Bases answers on authoritative sources.
- **Customizable:** Control which external sources it uses.
- **Cost-Effective:** Reduces training time and expense.

RAG Applications

- Conversational AI / Chatbots
- Enterprise knowledge search
- Customer support automation
- Healthcare literature lookup
- Legal & compliance document retrieval
- Research assistance
- Content generation with factual grounding



Newton School of Technology

Please fill the feedback form.